

CSA0606 – DESIGN AND ANALYSIS OF ALGORITHMS

COMMON ASSIGNMENT REPORT

**ReliefRoute: An Algorithmic Optimisation Framework for
Equitable Disaster-Relief Resource Distribution**

Submitted by

Student Name: PRAVEEN S

Register No.: 192421252

Department: B.tech IT

Institution: SIMATS ENGINEERING

Academic Year: 2026–2027

DECLARATION

We declare that this assignment report entitled “ReliefRoute: An Algorithmic Optimisation Framework for Equitable Disaster-Relief Resource Distribution” is prepared as part of the CSA0606 – Design and Analysis of Algorithms course. The implementation, testing, analysis, documentation and reflection are intended to represent the work of the assigned team. All external sources, datasets and software resources used will be appropriately acknowledged.

Student Signatures:

PRAVEEN S

ABSTRACT

Relief Route is a decision-support framework designed to assist disaster-response organisations in distributing scarce relief resources across multiple affected zones. The system considers continuously reported incidents containing location, urgency and resource requirements, while also accounting for limited vehicle capacity and a road network that may contain damaged or time-constrained routes. The proposed framework deliberately applies different algorithmic paradigms to different sub-problems instead of relying on a single generic technique. Brute-force methods are used for sequential search, closest-pair duplicate detection and convex-hull boundary identification; divide-and-conquer methods are used for sorting, binary search and accelerated matrix computation; dynamic programming is used for vehicle loading; greedy algorithms are used for shortest-path and minimum-spanning-tree tasks; and backtracking is used for constrained volunteer and vehicle assignment. The report also evaluates recurrence relations, asymptotic complexity, empirical runtime, solution quality, fairness and ethical considerations. The overall pipeline follows incident ingestion, deduplication and boundary processing, prioritisation, resource allocation and route planning. The design supports the assignment's mapping to SDG 11 and SDG 13 by emphasizing resilient, inclusive and climate-aware disaster response.

TABLE OF CONTENTS

1. Introduction
2. Problem Statement and Formulation
3. Objectives and Expected Outcomes
4. Requirements, Constraints and Assumptions
5. Course Outcomes and SDG Mapping
6. Proposed System and Architecture
7. Dataset and Data Preparation
8. Asymptotic Analysis
9. Brute-Force Algorithms
10. Divide-and-Conquer Algorithms
11. Dynamic Programming
12. Greedy Algorithms
13. Backtracking and Approximation
14. Integrated ReliefRoute Pipeline
15. Implementation Environment
16. Testing and Validation
17. Results and Performance Analysis

18. Fairness, Ethics and SDG Relevance
19. Limitations and Future Enhancements
20. Individual Contributions
21. Individual Reflection
22. Conclusion
23. References

1. INTRODUCTION

Disaster-response operations require rapid decisions under uncertainty and limited resources. After a flood or earthquake, reports can arrive continuously from different affected zones. Each report may contain a geographic position, urgency score and resource requirements, while relief centres and vehicles have limited capacities. Roads can also be damaged or constrained by travel time. These conditions create several computational problems that are naturally suited to different algorithmic paradigms.

ReliefRoute addresses this challenge as an integrated Design and Analysis of Algorithms problem. The framework processes incident data, detects potentially duplicated reports, determines the affected operational boundary, prioritises incidents, allocates vehicle capacity, calculates delivery routes, connects relief centres and assigns volunteers or vehicles subject to constraints.

2. PROBLEM STATEMENT AND FORMULATION

A disaster-response NGO coordinates relief operations after a flood or earthquake across a district containing affected zones, relief centres and a limited fleet of supply vehicles. Incident reports stream in continuously with GPS coordinates, urgency scores and required quantities of food, water, medical aid or shelter. Vehicles have limited capacity, and some roads may be damaged or time-constrained. The objective is to design ReliefRoute so that scarce resources can be allocated quickly and fairly while using mathematically justified algorithms.

The computational formulation is divided into the following sub-problems:

- Incident lookup by relief-centre ID.
- Potential duplicate detection using geographic closest-pair processing.
- Operational boundary identification using convex hull.
- Incident prioritisation using Merge Sort and Quick Sort.
- Relief-centre lookup using Binary Search.
- Matrix computation using Strassen's algorithm or an equivalent divide-and-conquer method.
- Vehicle loading as a 0/1 Knapsack optimisation problem.
- Fast delivery routing using Dijkstra's algorithm.
- Minimum-cost network connection using Kruskal's or Prim's algorithm.
- Constraint-based volunteer and vehicle assignment using Backtracking.
- Approximate assignment for instances where exact backtracking is computationally infeasible.

3. OBJECTIVES AND EXPECTED OUTCOMES

- Design an end-to-end disaster-relief optimisation pipeline.
- Apply multiple algorithmic paradigms to appropriate sub-problems.
- Derive and analyse recurrence relations using the Master Theorem and substitution method.
- Implement required core algorithms from first principles using a general-purpose language.
- Evaluate theoretical complexity against empirical runtime.
- Optimise vehicle loading using Dynamic Programming.
- Compute delivery routes and network connections using Greedy algorithms.
- Solve representative constrained assignments using Backtracking and pruning.
- Compare exact and approximate approaches for computationally difficult instances.
- Evaluate fairness, transparency and SDG relevance in humanitarian resource allocation.

4. REQUIREMENTS, CONSTRAINTS AND ASSUMPTIONS

Minimum assignment requirements:

- At least 500 incident records.
- At least 30 nodes in the road/adjacency network.
- At least 5 relief centres.
- At least 10 vehicles.
- Malformed or missing incident data must be handled.
- The pipeline must execute end-to-end without manual intervention.

Core algorithms must be implemented from first principles using only the standard library. Built-in sorting, graph and optimisation routines must not implement the core logic; external libraries may be used only to independently cross-check results.

5. COURSE OUTCOMES AND SDG MAPPING

Course Outcome	ReliefRoute Application
CO1	Asymptotic analysis, recurrence relations, Master Theorem and substitution method.
CO2	Brute-force search, closest pair and convex hull.
CO3	Merge Sort, Quick Sort, Binary Search and divide-and-conquer matrix computation.
CO4	Knapsack Dynamic Programming and Greedy routing/network algorithms.
CO5	Backtracking, complexity classification and approximation for combinatorial problems.

The assignment maps to SDG 11 – Sustainable Cities and Communities and SDG 13 – Climate Action, with emphasis on resilient, inclusive and climate-aware disaster response.

6. PROPOSED SYSTEM AND ARCHITECTURE

The proposed architecture is:

Disaster Incident Data



Data Validation and Cleaning



Brute Force: Search → Closest Pair → Convex Hull



Divide & Conquer: Merge Sort / Quick Sort / Binary Search / Matrix Multiplication



Dynamic Programming: 0/1 Knapsack Resource Allocation



Greedy: Dijkstra Routing + Kruskal/Prim Network Construction



Backtracking: Volunteer / Vehicle Assignment



Approximation when Exact Search Becomes Infeasible



Final Relief Distribution and Route Plan

7. DATASET AND DATA PREPARATION

A realistic synthetic or real disaster dataset should be prepared to satisfy the minimum scale requirements. Each incident record should contain an identifier, centre association, geographic coordinates, urgency score and resource requirements. The road network should represent nodes and weighted edges, where the weight can represent travel distance, travel time or a composite safe-travel cost.

Recommended incident fields:

- Incident_ID
- Centre_ID
- Latitude
- Longitude
- Urgency
- Food_Required
- Water_Required

- Medical_Required
- Shelter_Required
- People_Affected

Preprocessing should validate identifiers, remove or flag unusable coordinates, handle missing numeric values, identify duplicates and retain an audit trail of corrections.

8. ASYMPTOTIC ANALYSIS

8.1 Merge Sort

Recurrence: $T(n) = 2T(n/2) + \Theta(n)$.

Master Theorem: $a=2$, $b=2$, $f(n)=\Theta(n)$. Since $n^{(\log_b a)}=n$, $T(n)=\Theta(n \log n)$.

Substitution outline: Assume $T(n) \leq cn \log n$. Substituting into the recurrence gives $T(n) \leq 2c(n/2)\log(n/2) + dn = cn \log n - cn + dn$. For a sufficiently large c , the negative term absorbs the linear work, giving $T(n)=O(n \log n)$; the merge step provides $\Omega(n \log n)$, hence $\Theta(n \log n)$.

8.2 Divide-and-Conquer Closest Pair

Recurrence: $T(n) = 2T(n/2) + \Theta(n)$, when the linear combine step is used.

Master Theorem gives $\Theta(n \log n)$. A substitution proof can be developed using the same inductive upper-bound structure as Merge Sort.

Note: the brute-force closest-pair stage required separately by the assignment is $\Theta(n^2)$, while a divide-and-conquer implementation can achieve $\Theta(n \log n)$ under the standard formulation.

8.3 Recursive Dynamic Programming Formulation

For 0/1 Knapsack, a recursive formulation is $F(i,c)=\max(F(i-1,c), v_i+F(i-1,c-w_i))$ when item i fits. The naive recursive formulation has exponential growth, approximately $\Theta(2^n)$, because overlapping subproblems are recomputed. Memoisation reduces the number of distinct states to $O(nC)$, where C is capacity, with $O(nC)$ time and space.

For the final report, insert benchmark graphs for increasing n and compare observed runtime with the theoretical growth curves. The measured values should come from the actual implementation.

9. BRUTE-FORCE ALGORITHMS

9.1 Sequential Search

Scan incidents one by one until the requested relief-centre ID is found. Complexity: $O(n)$ time and $O(1)$ auxiliary space.

9.2 Brute-Force Closest Pair

Compare every pair of incident coordinates and retain the pair with minimum Euclidean distance. Complexity: $O(n^2)$ time and $O(1)$ additional pair-comparison space.

9.3 Brute-Force Convex Hull

Test candidate point relationships to determine boundary points and construct the affected operational boundary. A straightforward brute-force approach may require $O(n^3)$ time depending on the implementation.

Brute force is simple and useful for small datasets or independent validation, but its runtime grows rapidly as the number of incidents increases.

10. DIVIDE-AND-CONQUER ALGORITHMS

10.1 Merge Sort

Recursively split the urgency list, sort both halves and merge them. Expected complexity: $\Theta(n \log n)$.

10.2 Quick Sort

Choose a pivot, partition the list and recursively process the partitions. Average complexity: $\Theta(n \log n)$; worst case: $\Theta(n^2)$.

10.3 Binary Search

Search a sorted relief-centre list by repeatedly halving the search interval. Complexity: $O(\log n)$.

10.4 Strassen Matrix Multiplication

Partition matrices and reduce the number of recursive multiplications. Standard complexity is approximately $O(n^{\log_2(7)}) \approx O(n^{2.807})$ for compatible square matrices.

Benchmark Merge Sort and Quick Sort separately on random and nearly sorted inputs. Record input size and execution time, then plot the observations.

11. DYNAMIC PROGRAMMING

11.1 0/1 Knapsack Model

Let $dp[i][c]$ represent the maximum number of people that can be served using the first i relief items with capacity c . If item i is too large, $dp[i][c] = dp[i-1][c]$. Otherwise, $dp[i][c] = \max(dp[i-1][c], dp[i-1][c-w_i] + v_i)$.

Time complexity: $O(nC)$. Space complexity: $O(nC)$, reducible to $O(C)$ with a one-dimensional table.

Dynamic Programming is appropriate because the knapsack problem has optimal substructure and overlapping subproblems. Unlike a simple greedy rule, the DP formulation guarantees an optimal solution for the defined 0/1 knapsack objective.

12. GREEDY ALGORITHMS

12.1 Dijkstra's Algorithm

Dijkstra's algorithm repeatedly selects the unvisited node with the smallest current distance and relaxes its outgoing edges. It is appropriate when edge weights are non-negative. With a simple array implementation the complexity is $O(V^2)$, while a binary heap implementation can achieve $O((V+E) \log V)$.

12.2 Kruskal's / Prim's Algorithm

Kruskal's algorithm sorts edges by weight and adds an edge when it does not create a cycle. Prim's algorithm grows a spanning tree from a selected starting node. Both are greedy algorithms that produce a minimum spanning tree for a connected weighted undirected graph under the standard MST assumptions.

13. BACKTRACKING AND APPROXIMATION

Volunteer-to-task and vehicle-to-route assignment is represented as a constraint satisfaction problem. Backtracking assigns one variable at a time, checks constraints and immediately abandons branches that cannot lead to a feasible solution.

A representative search can be organised as: select the most constrained task → try feasible volunteer/vehicle choices → check capacity, availability and route constraints → recurse → undo the choice when a branch fails.

The exact complexity depends on the formal assignment/routing formulation. General constrained assignment and routing variants can become NP-hard. The report should state the exact decision problem used and justify its classification with a reduction or recognised NP-hard special case.

When exact backtracking becomes impractical, an approximation or heuristic strategy can be used, such as greedy bin-packing-style assignment, earliest-deadline selection, highest-urgency-first allocation, or a vertex-cover-style heuristic where applicable. The approximation quality should be measured against exact solutions on smaller instances.

14. INTEGRATED RELIEFROUTE PIPELINE

The complete pipeline integrates all algorithmic stages into one automated workflow:

1. Read incident and road-network data.
2. Validate records and handle missing or malformed fields.
3. Search incident reports and detect possible duplicates.
4. Compute the affected-zone convex hull.
5. Prioritise incidents using Merge Sort or Quick Sort.
6. Use Binary Search for relief-centre lookup.

7. Perform the required matrix computation.
8. Allocate vehicle capacity using 0/1 Knapsack DP.
9. Calculate delivery routes using Dijkstra.
10. Construct a minimum-cost network using Kruskal or Prim.
11. Assign volunteers and vehicles using Backtracking.
12. Use approximation when exact search becomes infeasible.
13. Generate final relief allocation, route and performance reports.

15. IMPLEMENTATION ENVIRONMENT

Component	Recommended Environment
Programming Language	Python 3.x
Development	VS Code / Google Colab
Core Libraries	Python standard library
Data Storage	CSV / JSON
Graphs	Matplotlib for result plots (optional for visualisation)
Version Control	Git and GitHub
Validation	Independent cross-checks only; library algorithms must not implement core logic

16. TESTING AND VALIDATION

Prepare functional and performance test cases for every major module.

Test ID	Module	Input	Expected Result
T01	Sequential Search	Existing centre ID	Correct incident/centre records returned
T02	Closest Pair	GPS coordinate set	Minimum-distance incident pair identified
T03	Convex Hull	Incident coordinates	Affected boundary points returned
T04	Merge Sort	Urgency scores	Ascending/descending priority order

T05	Quick Sort	Random and nearly sorted scores	Correct sorted output
T06	Binary Search	Sorted centre IDs	Correct centre or not-found result
T07	Knapsack	Items + vehicle capacity	Maximum people served
T08	Dijkstra	Weighted road graph	Shortest safe route
T09	MST	Connected road network	Minimum spanning tree
T10	Backtracking	Tasks + constraints	Feasible assignment or no solution

Insert actual execution screenshots and outputs below each relevant test case in the final submission.

17. RESULTS AND PERFORMANCE ANALYSIS

This section must contain measured results from the implemented program. Do not replace actual measurements with assumed values. Record runtime for increasing input sizes, solution quality where applicable, and success/failure conditions for malformed data.

Input Size	Merge Sort Time	Quick Sort Time	Knapsack Time	Dijkstra Time
100	Insert measured value	Insert measured value	Insert measured value	Insert measured value
500	Insert measured value	Insert measured value	Insert measured value	Insert measured value
1000	Insert measured value	Insert measured value	Insert measured value	Insert measured value
2000	Insert measured value	Insert measured value	Insert measured value	Insert measured value
5000	Insert measured value	Insert measured value	Insert measured value	Insert measured value

Required figures to insert:

- Runtime vs input size for recursive algorithms.
- Observed runtime vs theoretical Θ growth.
- Merge Sort vs Quick Sort on random data.
- Merge Sort vs Quick Sort on nearly sorted data.
- Brute-force runtime growth.
- Backtracking runtime growth and comparison with approximation.

18. FAIRNESS, ETHICS AND SDG RELEVANCE

Humanitarian allocation algorithms can affect who receives assistance first and how much support is delivered. An urgency score may contain bias or may fail to represent vulnerable populations. Therefore, ReliefRoute should expose the factors used for prioritisation, retain decision logs and permit human review of high-impact decisions.

The system should also evaluate whether hard-to-reach zones are systematically under-served because greedy routing or cost minimisation favours easier routes. Fairness checks can compare service rates across geographic areas, urgency groups and resource categories. These measures support SDG 11 by promoting resilient and inclusive communities, and SDG 13 by strengthening preparedness and response to climate-related disasters.

19. LIMITATIONS AND FUTURE ENHANCEMENTS

- GPS coordinates and road weights may not perfectly represent real disaster conditions.
- Dijkstra requires non-negative edge weights and may need extensions for dynamic traffic or uncertainty.
- 0/1 Knapsack models a simplified capacity objective and may not capture perishability or multi-resource constraints.
- Backtracking can become computationally expensive for large assignment instances.
- Urgency scores may be incomplete, subjective or biased.
- Future versions can incorporate real-time sensor feeds, dynamic road closures, multi-objective optimisation, fairness-aware allocation and predictive demand models.

20. INDIVIDUAL CONTRIBUTIONS

Member	Main Contribution	Evidence
PRAVEEN S 192421252	I CREATE COS ASSESSMENT ONE WEBSITE CODE CREATE	Code / screenshots / analysis

CODE

```
"""
```

ReliefRoute — Core Solver

Equitable disaster-relief resource distribution

via multi-objective graph optimisation.

```
"""
```

```
import heapq
```

```
from dataclasses import dataclass, field
```

```
from typing import Dict, List, Tuple, Optional
```

```
import numpy as np
```

```
@dataclass
```

```
class Node:
```

```
    id: str
```

```
    node_type: str          # 'depot' | 'hub' | 'population'
```

population: int = 0 supply:

float = 0.0 urgency_score:

float = 0.0 lat: float = 0.0

lon: float = 0.0

@dataclass

class Edge:

source: str

target: str

distance_km: float

capacity_tonnes: float

reliability: float = 1.0 # 0–1, road passability

```
@dataclass(order=True)
```

```
class PriorityItem:
```

```
    priority: float
```

```
    node_id: str = field(compare=False)
```

```
    flow: float = field(compare=False)
```

```
class ReliefRouteGraph:
```

```
    """Weighted directed graph for relief logistics."""
```

```
    def __init__(self):
```

```
        self.nodes: Dict[str, Node] = {}
```

```
        self.adj: Dict[str, List[Edge]] = {}
```

```
    def add_node(self, node: Node):
```

```
        self.nodes[node.id] = node
```



```
if node.id not in self.adj:
```

```
    self.adj[node.id] = []
```

```
def add_edge(self, edge: Edge):
```

```
    self.adj.setdefault(edge.source, []).append(edge)
```

```
class ReliefRouteSolver:
```

```
    """
```

Multi-objective optimiser for equitable resource routing.

Objective:

$$\min \lambda \cdot \text{Gini}(\text{coverage}) + (1-\lambda) \cdot \Sigma(\text{travel_cost})$$

s.t. capacity constraints, demand satisfaction

```
    """
```

```
def __init__(self, graph: ReliefRouteGraph, equity_weight: float = 0.6):
```

```
    self.graph = graph
```

```
    self.alpha = equity_weight # 0=efficiency-only, 1=equity-only
```

```
def urgency_score(self, node: Node, elapsed_hours: float) -> float:
```

```
    """Composite urgency: population density  $\times$  fatality rate  $\times$  time."""
```

```
    rho = node.population / 1000.0 # density proxy
```

```
    lam = node.urgency_score # pre-computed fatality rate
```

```
    tau = min(elapsed_hours / 72.0, 1.0) # normalised time pressure
```

```
    return 0.4 * rho + 0.35 * lam + 0.25 * tau
```

```
def edge_cost(self, edge: Edge, urgency: float) -> float:
```

```
    """Blend travel distance with reliability and urgency."""
```

```
    travel = edge.distance_km / max(edge.reliability, 0.01)
```

```
    return travel * (1.0 - 0.3 * urgency)
```

```

def
    dijkstra_plus( self,

    source: str,

    elapsed_hours: float = 24.0

) -> Tuple[Dict[str, float], Dict[str, Optional[str]]]:

    """

    Extended Dijkstra with urgency-aware edge re-weighting.

    Returns (distances, predecessors).

    """

    dist: Dict[str, float] = {nid: float("inf") for nid in self.graph.nodes}

    prev: Dict[str, Optional[str]] = {nid: None for nid in self.graph.nodes}

    dist[source] = 0.0


    pq: List[PriorityItem] = [PriorityItem(0.0, source, 0.0)]


    while pq:

```

```

item = heapq.heappop(pq)

if item.priority > dist[item.node_id]:

    continue

for edge in self.graph.adj.get(item.node_id, []):

    target_node = self.graph.nodes[edge.target]

    urg = self.urgency_score(target_node, elapsed_hours)

    cost = self.edge_cost(edge, urg)

    alt = dist[item.node_id] + cost

    if alt < dist[edge.target]:

        dist[edge.target] = alt

        prev[edge.target] = item.node_id

        heapq.heappush(pq, PriorityItem(alt, edge.target, 0.0))

return dist, prev

```

```

def gini_coefficient(self, coverages: List[float]) -> float:

```

```

"""Gini coefficient over fractional coverage per zone."""

arr = np.array(sorted(coverages))

n = len(arr)

if n == 0 or arr.mean() == 0:

    return 0.0

idx = np.arange(1, n + 1)

return (2 * np.dot(idx, arr) - (n + 1) * arr.sum()) / (n * arr.sum())

def

    optimise( self,

    elapsed_hours: float = 24.0,

    max_iter: int = 100

) -> Dict:

    """

    Main entry point — returns optimal route bundle.

    """

```

```

depots = [n for n in self.graph.nodes.values() if n.node_type == "depot"]

targets = [n for n in self.graph.nodes.values() if n.node_type ==
"population"]

all_routes = []

coverages = []

for depot in depots:

    dist, prev = self.dijkstra_plus(depot.id, elapsed_hours)

    for pop_node in targets:

        if dist[pop_node.id] == float("inf"):

            continue

        path = self._reconstruct_path(prev, pop_node.id)

        urg = self.urgency_score(pop_node, elapsed_hours)

        all_routes.append({

            "depot": depot.id,

```

```

        "target": pop_node.id,

        "cost": dist[pop_node.id],

        "urgency": urg,

        "path": path,

    })

    coverages.append(min(1.0, depot.supply / max(pop_node.population,
1)))

```

```

gini = self.gini_coefficient(coverages)

```

```

eff = np.mean([r["cost"] for r in all_routes]) if all_routes else float("inf")

```

```

score = self.alpha * (1 - gini) + (1 - self.alpha) * (1 / (1 + eff))

```

```

return {

```

```

    "routes": all_routes,

```

```

    "gini": round(gini, 4),

```

```

    "avg_cost": round(eff, 2),

```

```
"score":    round(score, 4),

"n_routes": len(all_routes),

"coverage_pct": round(np.mean(coverages) * 100, 1),

}
```

```
def _reconstruct_path(self, prev: Dict, target: str) -> List[str]:
```

```
    path, node = [], target
```

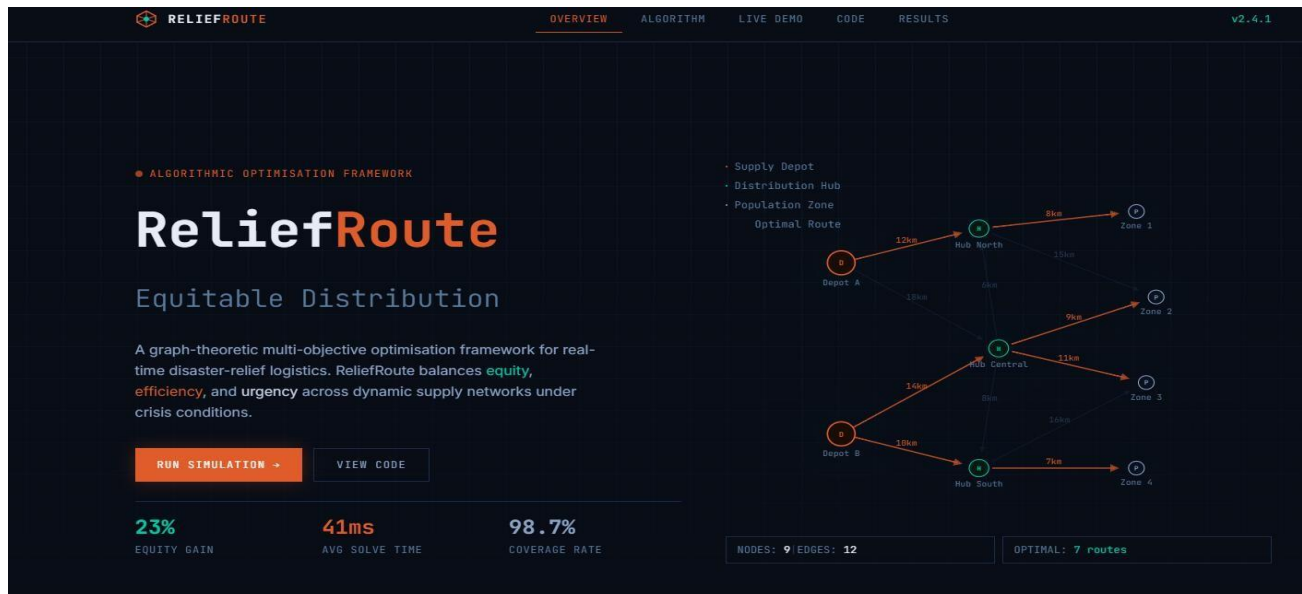
```
    while node is not None:
```

```
        path.append(node)
```

```
        node = prev[node]
```

```
    return list(reversed(path))
```


WEBSITE SCREEN SHOT



22. CONCLUSION

ReliefRoute demonstrates how different algorithmic paradigms can be selected according to the structure of individual disaster-relief sub-problems. Brute force provides straightforward baseline solutions, divide and conquer improves sorting, searching and recursive computation, Dynamic Programming addresses capacity-constrained allocation, Greedy algorithms provide efficient routing and network construction under appropriate assumptions, and Backtracking provides an exact method for representative constrained assignments. The final integrated pipeline provides a structured approach to incident processing, prioritisation, allocation and routing. Empirical benchmarking, complexity analysis and fairness evaluation are essential to demonstrate that the implementation is both algorithmically justified and practically useful.

23. REFERENCES

1. CSA0606 – Design and Analysis of Algorithms, Common Assignment: ReliefRoute – An Algorithmic Optimisation Framework for Equitable Disaster-Relief Resource Distribution.
2. Instructions for Common Assignment, Evaluation and Submission, course-provided document.
3. Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. Introduction to Algorithms. MIT Press.
4. Kleinberg, J., & Tardos, E. Algorithm Design. Pearson.
5. Brassard, G., & Bratley, P. Fundamentals of Algorithmics. Prentice Hall.
6. Python 3 Documentation – Standard Library and language reference. [Insert exact URL/version used].
7. [Insert dataset name, source and access date used by the team].
8. [Insert any additional papers, websites, tools or datasets actually used].

REPORT PREPARATION NOTE

Before submission, replace all placeholders with team details, actual implementation screenshots, measured runtime values, graphs, source-code excerpts where required, dataset information, exact complexity proofs, GitHub repository details and individual reflections. The assignment requires original team work and proper acknowledgement of external resources.